School of Engineering Mathematics and Technology

**SEMT20001: Principles of Computational Modelling**

**COURSEWORK ASSESSMENT: QUESTION 1**

**The question below forms one part of the assessment for the unit.**
**This is question 1 of 4.**
**Each question carries 25 marks. Answer all questions.**
**The maximum mark for this assessment is 100 marks.**
The deadline for submission for all questions is **1pm on 19 March 2026 (Week 21)**.

You must submit **two** items (on Blackboard) for **each** question:

(1) a single self-contained Jupyter Notebook, that reports your answers to the question, and explains how you found them, including all relevant Python commands and functions, and explanations of your code design, **and**

(2) a single .zip file containing all Python .py files.

Your Jupyter Notebook (1) for this question must contain everything you wish to be marked for this question (including, but not limited to, numerical and textual answers, figures, and all Python code used to generate them). Material not contained in your document (1) will receive no credit. Any Python .py files that are not contained in your .zip file (2) will receive no credit.

Your submission must run from the `pocm2526` environment, as defined on Blackboard (see *Learning and Teaching Area : Read me first! Python starter guide* for details). Any code that does not will receive no credit.

Your submission must be your own individual effort; submissions will be checked for possible plagiarism. You may only use tools such as spelling and grammar checkers in this assignment, and their use should be limited to corrections of your own work rather than substantial re-writes or extended contributions. Use of AI applications (even if they are free) beyond this is contract cheating; see `https://www.bristol.ac.uk/students/support/academic-advice/academic-integrity/contract-cheating/`. This is an AI Category 2 assessment; see `https://www.bristol.ac.uk/bilt/sharing-practice/guides/guidance-on-ai/using-ai-in-assessment/` for full details.

## Question 1: Computability & Complexity

For the following questions all Python code must be documented so as to make clear the underlying purpose of different parts of the code. Failure to do so will result in marks being deducted. You must show that you have systematically tested all code to show that it is functioning as required.

(a) Design a Turing machine that inputs a sequence of $n$ 1s, followed by a 0, followed by a sequence of $m$ 1s and outputs a single sequence of $n+m$ 1$s$. The Turing machine should be initialised scanning

the left most 1 in the left sequence of 1s and terminate scanning the left most one in the sequence of $n + m$ 1s. You many only use the following actions: move left, move right, write 1, write 0 and terminate. Instructions should take the form (`current state`, `reading`, `action`, `next state`) and you should include an intuitive description of the purpose of different blocks of instructions.

*(5 marks)*

(b) Write a Python implementation of your Turing machine from Q1a. Your code should include a suitable mechanism for dealing with the fact that lists (representing the tape) cannot be infinite and therefore they need to grow in length as and when required. You should demonstrate your code working on at least two test cases.

*(4 marks)*

(c) Show that $\lceil \log_2(n+1) \rceil$ is a recursive function for $n \in \mathbb{N}$.

*(3 marks)*

(d) Binary search is an algorithm for finding the position of a specified target element in an ordered list of numbers. The algorithms is as follows: Given an ordered `list` of $n$ elements and `target` where `target` may or may not be an element of `list`:

- Step 1: let `lower` $= 0$ & `upper` $= n - 1$.
- Step 2: If `upper` $<$ `lower` then terminate and return `'fail'`. Else:
- Step 3: Let `middle` $= \lfloor \frac{\texttt{lower}+\texttt{upper}}{2} \rfloor$.
- Step 4: If `list`[`middle`] $=$ `target` then terminate and return `middle`. Else:
- Step 5: If `target` $<$ `middle` then let `upper` $=$ `middle` $- 1$. Else if `target` $>$ `middle` then `lower` $=$ `middle` $+ 1$.
- Step 6: Goto Step 2.

Write your own well documented Python code for `binarysearch` taking two inputs; an ordered list and a target element. Your implementation must recursively call the `binarysearch` function. Explain your design and implementation decisions, and any tests used to verify and validate the code.

*(4 marks)*

(e) Give an argument to show that the worst case time complexity of the binary search algorithm in Q1d is $O(\log_2(n))$.

*(3 marks)*

(f) Write Python code to evaluate the *average number of iterations*[1] taken by `binarysearch` from Q1d, as a function of list length. Assuming that the `target` is always in the `list`, by using this code and the `numpy` function `polyfit` run simulations to evaluate the hypothesis that the average time complexity of `binarysearch` is $O(\log_2(n))$.

*(6 marks)*

[1]Note that for this question you are being asked to count iterations not measure runtime.